
Data Structures

BS (CS) _Fall_2025

Lab_02 Manual



Learning Objectives:

1. Class Templates with unit testing (Gtest)

Class Templates in C++

A class template in C++ is a blueprint for creating classes that can work with any data type.

Just like function templates, class templates let us **“write once, use for all types,”** but instead of making a single generic function, we make a generic class.

When you create an object from a class template, the compiler generates a type-specific version of that class for the given type(s).

Anatomy of a Class Template

```
template <typename T1, typename T2>
class MyClass {
private:
    T1 first;
    T2 second;

public:
    MyClass(T1 a, T2 b) : first(a), second(b) {}
    void display() {
        cout << first << " " << second << endl;
    }
};
```

Breakdown:

- `template <typename T1, typename T2> :-` declares two type parameters.
- `T1 first; T2 second; :-` data members can be of any type.
- Constructor and methods can use these parameters.
- When we write `MyClass<int, string> obj(1, "Hello");` the compiler generates a class where `T1 = int` and `T2 = string`.

Multiple Template Parameters

Class templates can have more than one template parameter useful when types differ.

```
template <typename K, typename V>
class Pair {
    K key;
    V value;
};
```

Default Template Parameters

We can give default types for template parameters.

```
template <typename K = int, typename V = string>
class Dictionary {
    // ...
};
```

Now:

```
Dictionary<> d1;           // int → string
Dictionary<string> d2;     // string → string
Dictionary<string, double> d3; // string → double
```

Non-type Template Parameters

A non-type template parameter is a constant value (int, char, etc.) passed to a template.

Example:

```
template <typename T, int SIZE>
class FixedArray {
    T arr[SIZE];
};
```

Here, SIZE is fixed at compile time:

```
FixedArray<int, 5> nums; // array of 5 ints
```

Partial Specialization

Partial specialization allows customizing a class template for certain type combinations.

Example:

```
template <typename K, typename V>
class Dictionary {
    // generic version
};

template <typename V>
class Dictionary<string, V> {
    // specialized version for string keys
};
```

```
};
```

We can use this for Optimized behavior for certain types.